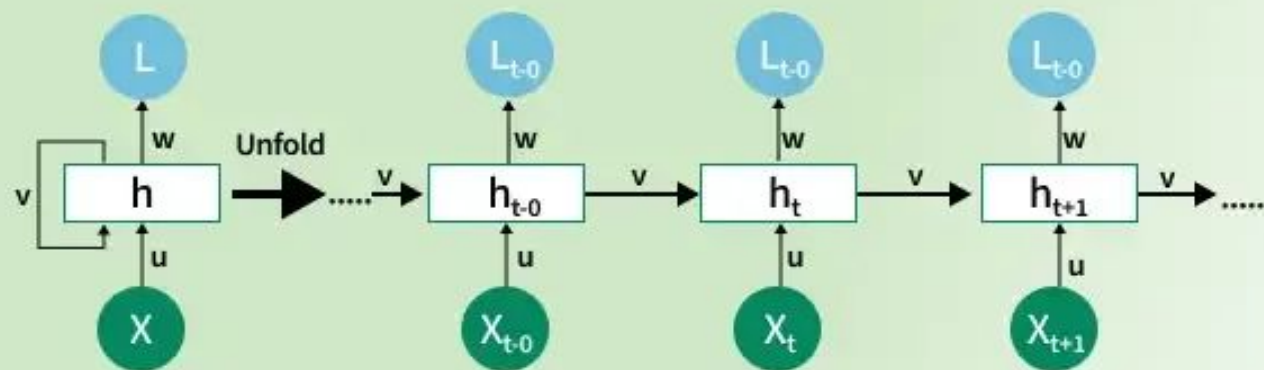
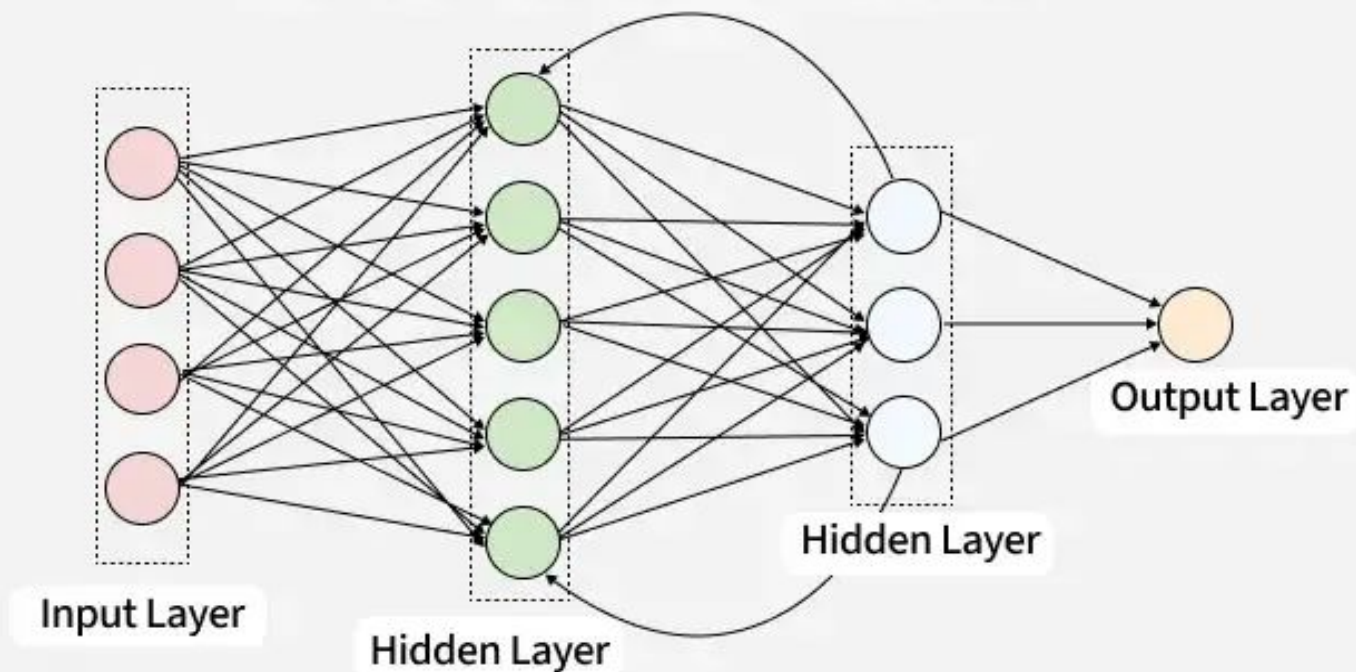
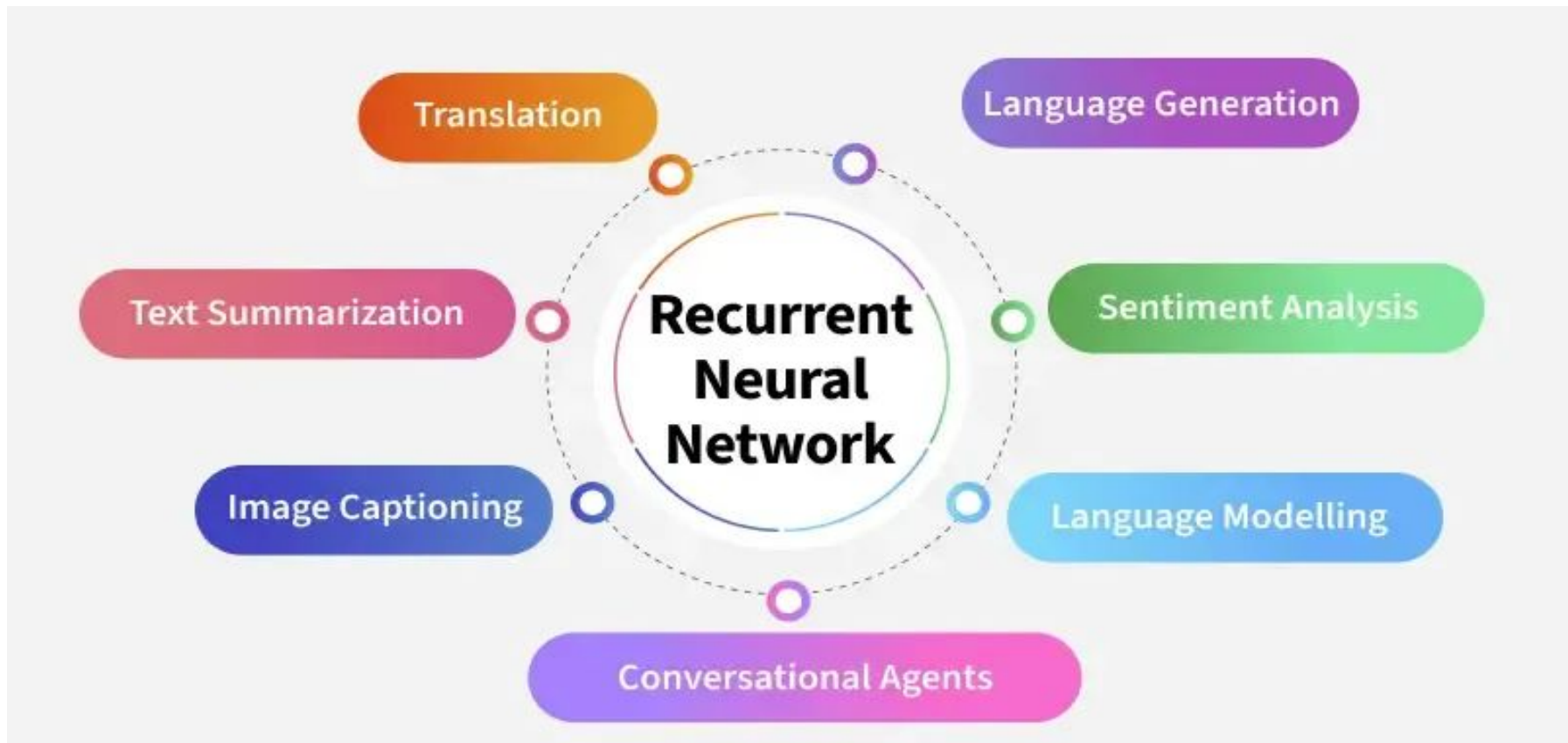


Introduction to Recurrent Neural Network



Recurrent Neural Network





Recurrent Neural Networks (RNNs)

differ from regular neural networks in how they process information. While standard neural networks pass information in one direction from input to output, RNNs feed information back into the network at each step.

Imagine reading a sentence and you try to predict the next word, you don't rely only on the current word but also remember the words that came before.

RNNs work similarly by “remembering” past information and passing the output from one step as input to the next i.e it considers all the earlier words to choose the most likely next word.

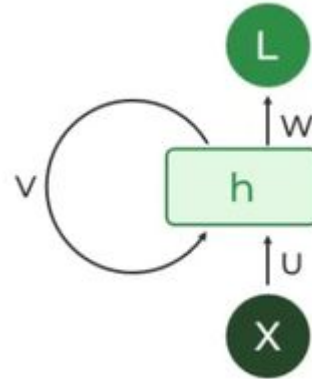
This memory of previous steps helps the network understand context and make better predictions.

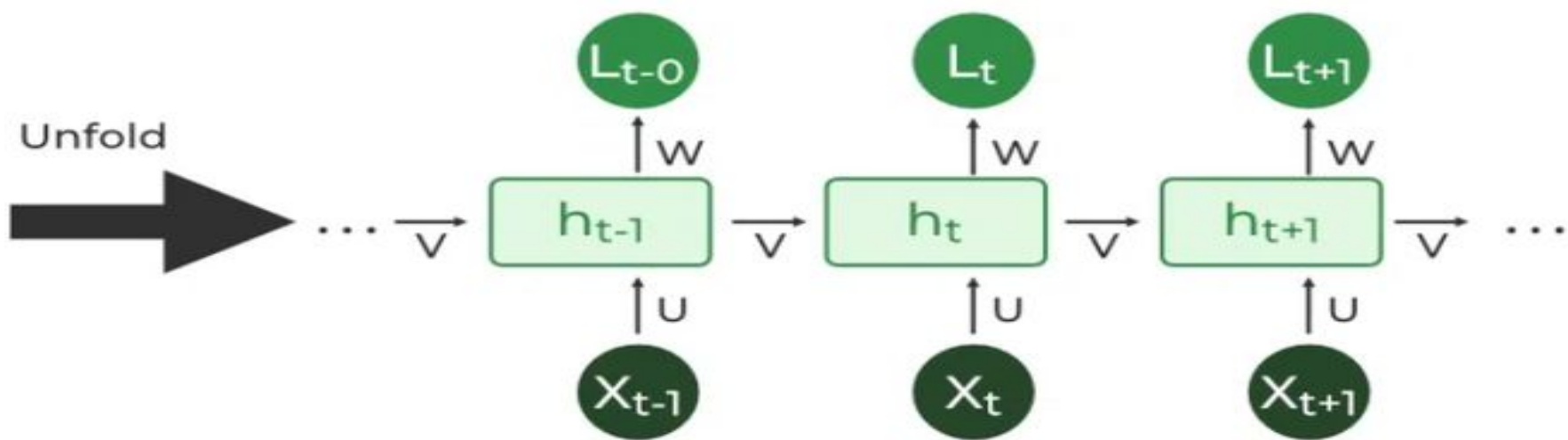
Recurrent Neurons

The fundamental processing unit in RNN is a **Recurrent Unit**.

They hold a hidden state that maintains information about previous inputs in a sequence.

Recurrent units can "remember" information from prior steps by feeding back their hidden state, allowing them to capture dependencies across time.





Recurrent Neural Network Architecture

RNNs share similarities in input and output structures with other deep learning architectures but differ significantly in how information flows from input to output. Unlike traditional deep neural networks where each dense layer has distinct weight matrices. RNNs use shared weights across time steps, allowing them to remember information over sequences.

In RNNs the hidden state H_i is calculated for every input X_i to retain sequential dependencies. The computations follow these core formulas:

1. Hidden State Calculation:

$$h = \sigma(U \cdot X + W \cdot h_{t-1} + B)$$

Here:

- h represents the current hidden state.
- U and W are weight matrices.
- B is the bias.

2. Output Calculation:

$$Y = O(V \cdot h + C)$$

The output Y is calculated by applying O an activation function to the weighted hidden state where V and C represent weights and bias.

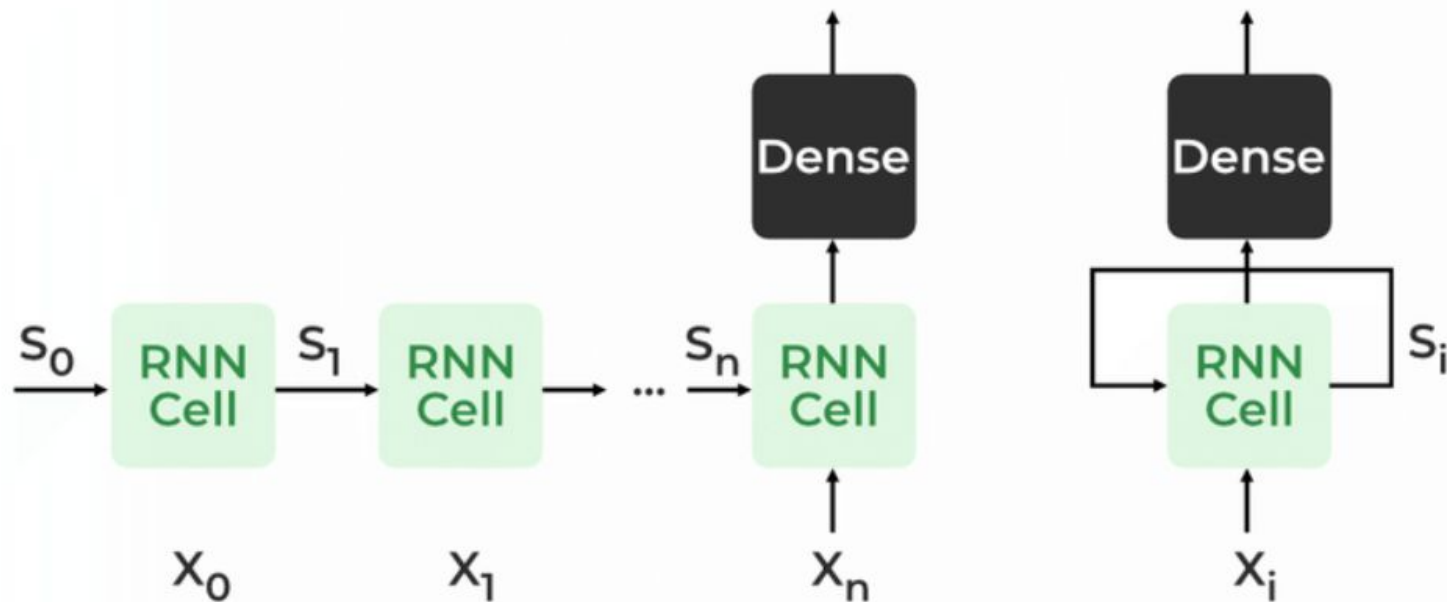
3. Overall Function:

$$Y = f(X, h, W, U, V, B, C)$$

This function defines the entire RNN operation where the state matrix S holds each element s_i representing the network's state at each time step i .

This function defines the entire RNN operation where the state matrix S holds each element s_i representing the network's state at each time step i .

RECURRENT NEURAL NETWORKS



How does RNN work?

At each time step RNNs process units with a fixed activation function. These units have an internal hidden state that acts as memory that retains information from previous time steps. This memory allows the network to store past knowledge and adapt based on new inputs.

Updating the Hidden State in RNNs

The current hidden state h_t depends on the previous state h_{t-1} and the current input x_t and is calculated using the following relations:

1. State Update:

$$h_t = f(h_{t-1}, x_t)$$

where:

- h_t is the current state
- h_{t-1} is the previous state
- x_t is the input at the current time step

2. Activation Function Application:

$$h_t = \tanh(W_{hh} \cdot h_{t-1} + W_{xh} \cdot x_t)$$

Here, W_{hh} is the weight matrix for the recurrent neuron and W_{xh} is the weight matrix for the input neuron.

3. Output Calculation:

$$y_t = W_{hy} \cdot h_t$$

where y_t is the output and W_{hy} is the weight at the output layer.

These parameters are updated using backpropagation. However, since RNN works on sequential data here we use an updated backpropagation which is known as **backpropagation through time**.

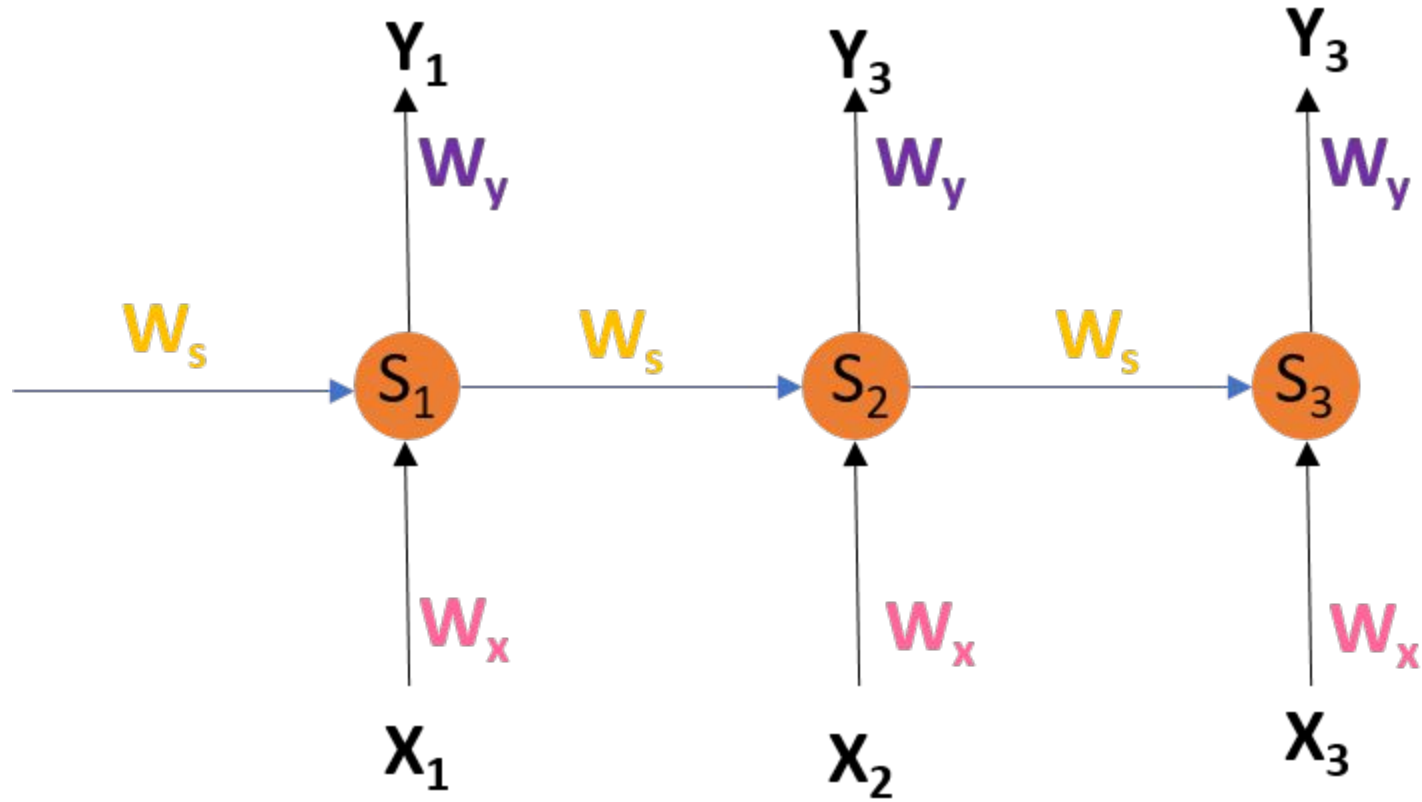
Recurrent Neural Networks (RNNs) are designed to process sequential data. Unlike traditional neural networks, RNN outputs depend not only on the current input but also on previous inputs through a memory element. This memory allows RNNs to capture temporal dependencies in data such as time series or language.

Training RNNs involves backpropagation where instead of updating weights based only on the current timestep

t , we also consider all previous timesteps :

$t-1, t-2, t-3, \dots$

This method is called Backpropagation Through Time (BPTT) which extends traditional backpropagation to sequential data by unfolding the network over time and summing gradients across all relevant time steps. This method enables RNNs to learn complex temporal patterns.



RNN Architecture

At each timestep t , the RNN maintains a hidden state S_t , which acts as the network's memory summarizing information from previous inputs. The hidden state S_t updates by combining the current input X_t and the previous hidden state S_{t-1} , applying an activation function to introduce non-linearity. Then the output Y_t is generated by transforming this hidden state.

$$S_t = g_1(W_x X_t + W_s S_{t-1})$$

- S_t represents the hidden state (memory) at time t .
- X_t is the input at time t .
- Y_t is the output at time t .
- W_s, W_x, W_y are weight matrices for hidden states, inputs and outputs, respectively.

$$Y_t = g_2(W_y S_t)$$

where g_1 and g_2 are activation functions.

Error Function at Time $t = 3$

To train the network, we measure how far the predicted output Y_t is from the desired output d_t using an error function. We use the squared error to measure the difference between the desired output d_t and actual output Y_t :

$$E_t = (d_t - Y_t)^2$$

At $t = 3$:

$$E_3 = (d_3 - Y_3)^2$$

This error quantifies the difference between the predicted output and the actual output at time 3.

Updating Weights Using BPTT

BPTT updates the weights W_y , W_s , W_x to minimize the error by computing gradients. Unlike standard backpropagation, BPTT unfolds the network across time steps, considering how errors at time t depend on all previous states.

We want to adjust the weights W_y , W_s and W_x to minimize the error E_3 .

1. Adjusting Output Weight W_y

The output weight W_y affects the output directly at time 3. This means we calculate how the error changes as Y_3 changes, then how Y_3 changes with respect to W_y . Updating W_y is straightforward because it only influences the current output.

Using the chain rule:

$$\frac{\partial E_3}{\partial W_y} = \frac{\partial E_3}{\partial Y_3} \times \frac{\partial Y_3}{\partial W_y}$$

- E_3 depends on Y_3 , so we differentiate E_3 w.r.t. Y_3 .
- Y_3 depends on W_y , so we differentiate Y_3 w.r.t. W_y .

2. Adjusting Hidden State Weight W_s

The hidden state weight W_s influences not just the current hidden state but all previous ones because each hidden state depends on the previous one. To update W_s , we must consider how changes to W_s affect all hidden states S_1, S_2, S_3 and consequently the output at time 3.

The gradient for W_s considers all previous hidden states because each hidden state depends on the previous one:

$$\frac{\partial E_3}{\partial W_s} = \sum_{i=1}^3 \frac{\partial E_3}{\partial Y_3} \times \frac{\partial Y_3}{\partial S_i} \times \frac{\partial S_i}{\partial W_s}$$

Breaking down:

- Start with the error gradient at output Y_3 .
- Propagate gradients back through all hidden states S_3, S_2, S_1 since they affect Y_3 .
- Each S_i depends on W_s , so we differentiate accordingly.

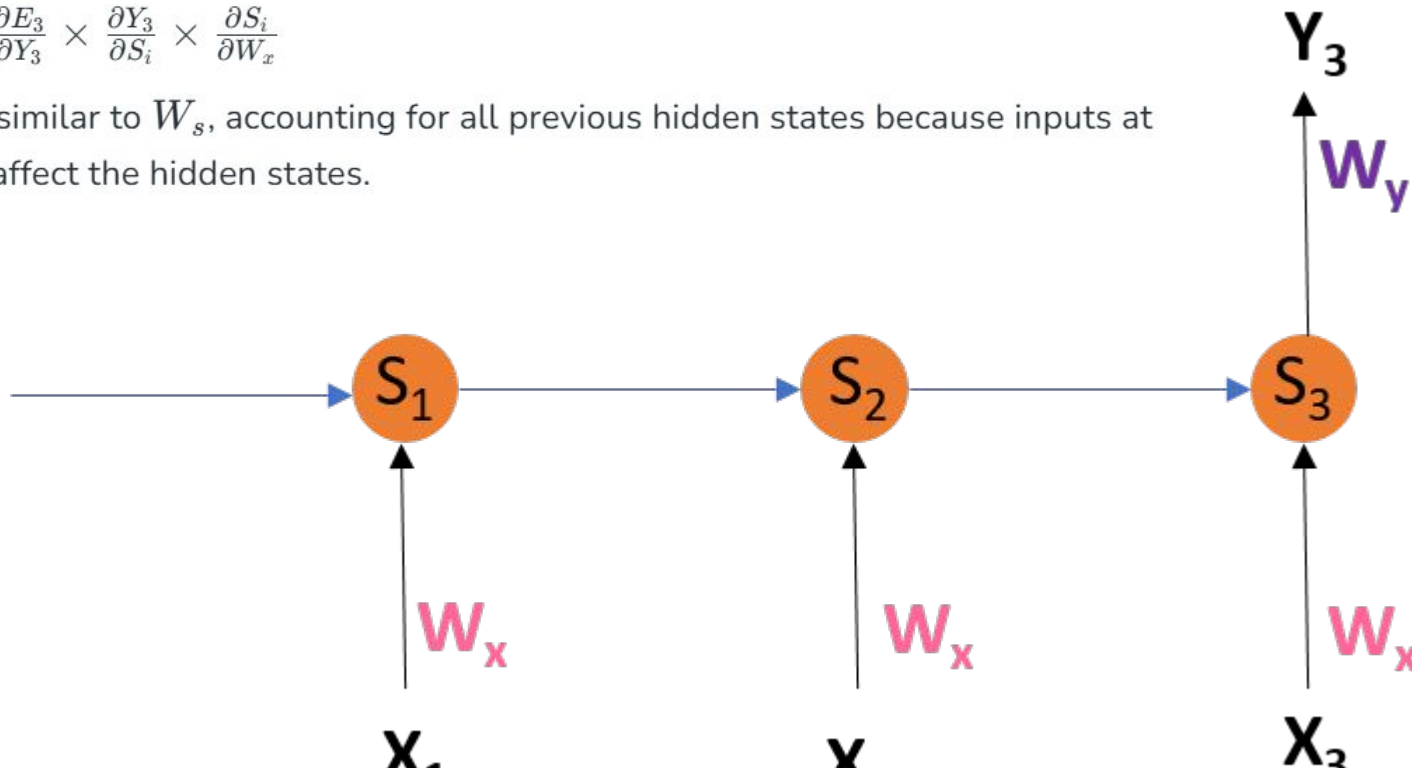


3. Adjusting Input Weight W_x

Similar to W_s , the input weight W_x affects all hidden states because the input at each timestep shapes the hidden state. The process considers how every input in the sequence impacts the hidden states leading to the output at time 3.

$$\frac{\partial E_3}{\partial W_x} = \sum_{i=1}^3 \frac{\partial E_3}{\partial Y_3} \times \frac{\partial Y_3}{\partial S_i} \times \frac{\partial S_i}{\partial W_x}$$

The process is similar to W_s , accounting for all previous hidden states because inputs at each timestep affect the hidden states.



Advantages of Backpropagation Through Time (BPTT)

- **Captures Temporal Dependencies:** BPTT allows RNNs to learn relationships across time steps, crucial for sequential data like speech, text and time series.
- **Unfolding over Time:** By considering all previous states during training, BPTT helps the model understand how past inputs influence future outputs.
- **Foundation for Modern RNNs:** BPTT forms the basis for training advanced architectures such as LSTMs and GRUs, enabling effective learning of long sequences.
- **Flexible for Variable Length Sequences:** It can handle input sequences of varying lengths, adapting gradient calculations accordingly.

Limitations of BPTT

- **Vanishing Gradient Problem:** When backpropagating over many time steps, gradients tend to shrink exponentially, making early time steps contribute very little to weight updates. This causes the network to “forget” long-term dependencies.
- **Exploding Gradient Problem:** Gradients may also grow uncontrollably large, causing unstable updates and making training difficult.

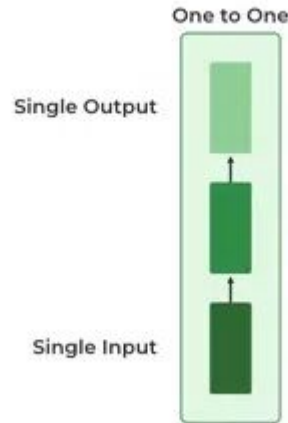
Solutions

- **Long Short-Term Memory (LSTM):** Special RNN cells designed to maintain information over longer sequences and mitigate vanishing gradients.
- **Gradient Clipping:** Limits the magnitude of gradients during backpropagation to prevent explosion by normalizing them when exceeding a threshold.

Types Of Recurrent Neural Networks

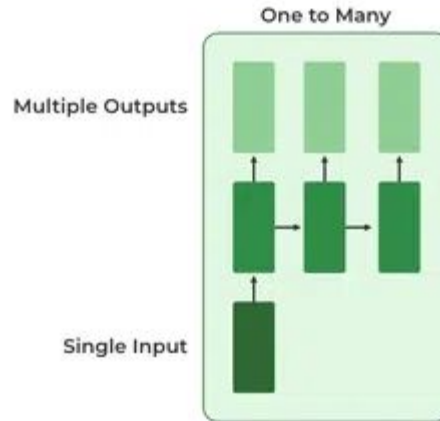
1. One-to-One RNN

This is the simplest type of neural network architecture where there is a single input and a single output. It is used for straightforward classification tasks such as binary classification where no sequential data is involved.



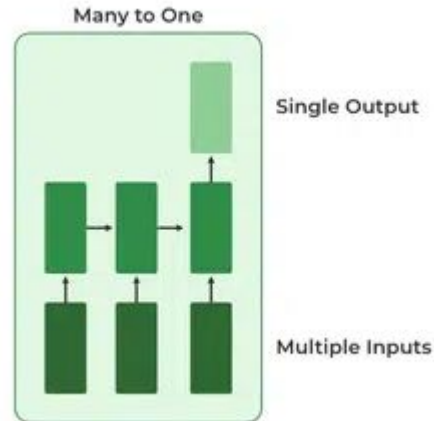
2. One-to-Many RNN

In a One-to-Many RNN the network processes a single input to produce multiple outputs over time. This is useful in tasks where one input triggers a sequence of predictions (outputs). For example in image captioning a single image can be used as input to generate a sequence of words as a caption.



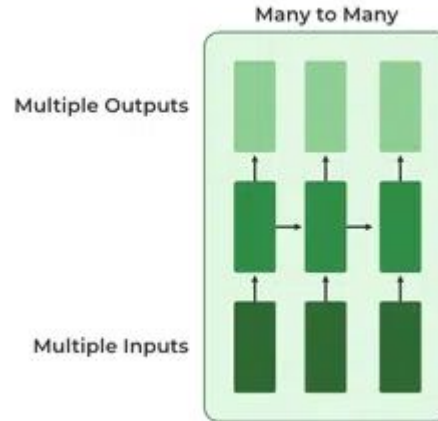
3. Many-to-One RNN

The **Many-to-One RNN** receives a sequence of inputs and generates a single output. This type is useful when the overall context of the input sequence is needed to make one prediction. In sentiment analysis the model receives a sequence of words (like a sentence) and produces a single output like positive, negative or neutral.



4. Many-to-Many RNN

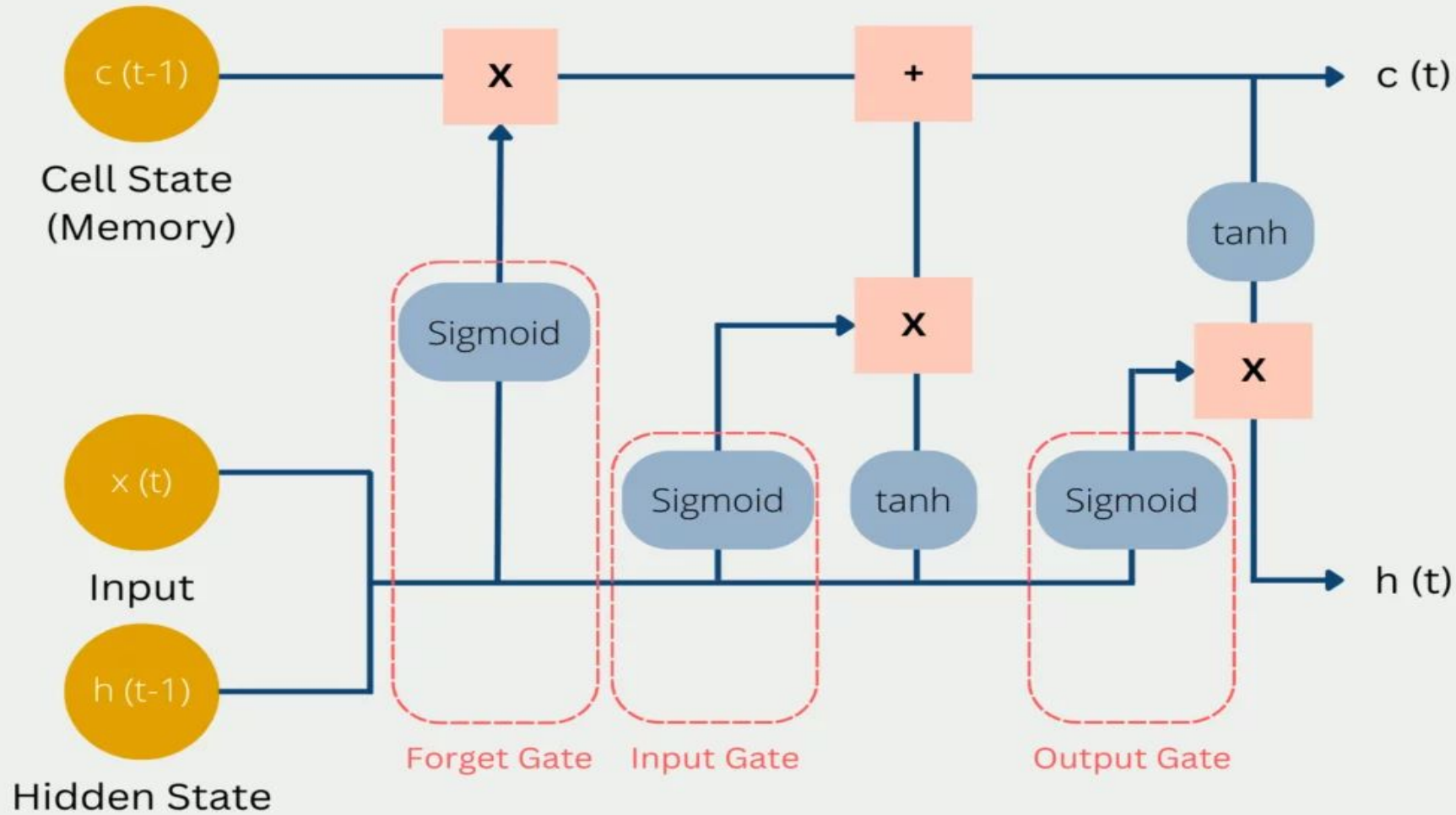
The **Many-to-Many RNN** type processes a sequence of inputs and generates a sequence of outputs. In language translation task a sequence of words in one language is given as input and a corresponding sequence in another language is generated as output.



LSTM

LSTM networks are an extension of recurrent neural networks (RNNs) mainly introduced to handle situations where RNNs fail.

- It fails to store information for a longer period of time. At times, a reference to certain information stored quite a long time ago is required to predict the current output. But RNNs are absolutely incapable of handling such “long-term dependencies”.
- There is no finer control over which part of the context needs to be carried forward and how much of the past needs to be ‘forgotten’.
- Other issues with RNNs are exploding and vanishing gradients (explained later) which occur during the training process of a network through backtracking.



. Forget Gate – “What should I throw away?”

- Looks at the **previous hidden state** and **current input**
- Outputs a number between 0 and 1 for each piece of memory
 - 0 → completely forget
 - 1 → completely keep
- This helps remove irrelevant information from the memory

Goal: Clean up old memory

Think: *“Is this still important?”*

Input Gate – “What new stuff should I store?”

- Again looks at the **previous hidden state** and **current input**
- Decides:
 - Which values to update (via a sigmoid gate)
 - What the new memory values should be (via a tanh layer)
- Updates the memory with **important new information**

Goal: Learn new information

Think: *“Is this new thing important enough to remember?”*

Cell State Update – “Update the memory”

- Combines:
 - What you **kept** from before (via forget gate)
 - What you **just learned** (via input gate)
- Result is the **new memory** (cell state)

Goal: Keep memory up to date

Think: *“My new memory is a mix of what I kept and what I learned.”*

Output Gate – “What should I show right now?”

- Decides what part of the memory (cell state) to show as output
- Again looks at **previous hidden state** and **current input**
- Final output is a filtered version of memory

Goal: Decide the current output (hidden state)

Think: *“What do I say or pass on right now?”*

1. Cell State (memory line)

- It's like a conveyor belt.
- It carries important information across the time steps.
- The LSTM can add or remove information from this state using gates.

2. Gates (control switches)

There are 3 main gates, each made of a sigmoid layer + pointwise multiplication:

a. Forget Gate

"What should I forget?"

- Input: Current input x_t + Previous hidden state h_{t-1}
- Output: A number between 0 (forget) and 1 (keep) for each value in the cell state

b. Input Gate

"What new info should I add?"

- Has two parts:
 1. Input gate layer (sigmoid): decides which values to update
 2. Candidate layer (tanh): creates new values to add

c. Output Gate

"What should I output?"

- Decides what the next hidden state h_t will be.
- This is used as output and passed to the next time step.